Automated As Built Documentation for BTSGSD Infrastructure

Generating Word PDF and Markdown deliverables with Azure DevOps

Implementation guide for the BTSGSD-Infrastructure engineering team

Prepared 11 November 2026

Purpose and scope

This guide describes a read-only documentation pipeline for the BTSGSD-Infrastructure repository. The pipeline authenticates to AWS, inventories selected resources, and generates Word, PDF, and Markdown as pipeline artifacts.

The initial implementation covers AWS Lambda and Amazon DynamoDB. It is deliberately separate from infrastructure deployment pipelines so that documentation generation cannot modify or redeploy resources.

At a glance

Stage	Technology	Outcome
Sources	AWS Cloud, Terraform, Azure Repos	Infrastructure context and automation source
Pipeline	Azure DevOps and Python 3.12	Authenticated read-only collection
Generator	boto3, python-docx, ReportLab	Word, PDF, and Markdown artifacts
Publishing	Pipeline artifacts and Azure Wiki	Downloadable reports and Wiki-ready content

2 Automated process

The process has three stages: infrastructure sources, pipeline execution, and artifact generation. AWS Cloud and Terraform provide the infrastructure context; Azure Repos stores the automation; Azure Pipelines runs Python; and the generator produces editable, review, and Wiki-ready formats.

3 Repository preparation

Create a documentation folder, place the generator inside it, and keep generated output outside source control. The root pipeline file orchestrates dependency installation, AWS authentication, generation, and artifact publication.

```
BTSGSD-Infrastructure/
|-- documentation/
|   |-- generate_asbuilt.py
|   `-- output/
|-- environments/
|   |-- dev/
|   `-- prod/
`-- asbuilt-pipeline.yml
```

4 Dependencies and permissions

The generator uses boto3 to read AWS configuration, python-docx to create Word documents, and ReportLab to create PDFs. The pipeline installs these libraries at runtime.

The AWS identity needs only read permissions for the implemented inventory. At minimum, allow sts:GetCallerIdentity, lambda:ListFunctions, and dynamodb:ListTables. Add read actions explicitly as additional services are introduced.

Library	Purpose
boto3	Read AWS resource configuration
python-docx	Create Microsoft Word documents
ReportLab	Create PDF reports

```
python -m pip install boto3 python-docx reportlab
```

5 Generator implementation

Create documentation/generate_asbuilt.py with the following working baseline. It paginates Lambda and DynamoDB listings and writes consistent Word, PDF, and Markdown outputs.

```
import argparse
import os
from datetime import datetime, timezone

import boto3
from docx import Document
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle


def collect_aws_resources(region):
    session = boto3.Session(region_name=region)
    account = session.client("sts").get_caller_identity()["Account"]
    resources = []

    lambda_client = session.client("lambda")
    for page in lambda_client.get_paginator("list_functions").paginate():
        for function in page.get("Functions", []):
            resources.append({
                "name": function["FunctionName"],
                "type": "AWS Lambda",
                "details": f"Runtime: {function.get('Runtime', 'N/A')}"
            })

    dynamodb = session.client("dynamodb")
    for page in dynamodb.get_paginator("list_tables").paginate():
        for table_name in page.get("TableNames", []):
            resources.append({
                "name": table_name,
                "type": "Amazon DynamoDB",
                "details": "DynamoDB table"
            })

    return account, sorted(resources, key=lambda item: (item["type"], item["name"]))


def create_word(path, title, metadata, resources):
    document = Document()
    document.add_heading(title, 0)
    for key, value in metadata.items():
        document.add_paragraph(f"{key}: {value}")
    document.add_heading("AWS Resource Inventory", level=1)
    table = document.add_table(rows=1, cols=3)
    table.style = "Table Grid"
    for cell, label in zip(table.rows[0].cells, ["Resource Name", "Type", "Details"]):
        cell.text = label
    for resource in resources:
        cells = table.add_row().cells
        cells[0].text = resource["name"]
        cells[1].text = resource["type"]
        cells[2].text = resource["details"]
    document.save(path)


def create_pdf(path, title, metadata, resources):
    styles = getSampleStyleSheet()
    story = [Paragraph(title, styles["Title"]), Spacer(1, 12)]
    for key, value in metadata.items():
        story.append(Paragraph(f"<b>{key}:</b> {value}", styles["BodyText"]))
    story.extend([Spacer(1, 12), Paragraph("AWS Resource Inventory", styles["Heading1"])])
    data = [["Resource Name", "Type", "Details"]]
    data.extend([[item["name"], item["type"], item["details"]] for item in resources])
    table = Table(data, repeatRows=1, colWidths=[190, 110, 190])
    table.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#17365D")),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
        ("GRID", (0, 0), (-1, -1), 0.5, colors.HexColor("#D9D9D9")),
        ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
        ("FONTSIZE", (0, 0), (-1, -1), 8),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#F3F6FA")]),
    ]))
    story.append(table)
    SimpleDocTemplate(path, pagesize=A4).build(story)


def create_markdown(path, title, metadata, resources):
    lines = [f"# {title}", ""]
    for key, value in metadata.items():
        lines.append(f"**{key}:** {value}  ")
    lines.extend(["", "## AWS Resource Inventory", "", "| Resource Name | Type | Details |", "|---|---|---|"])
    for resource in resources:
        values = [str(resource[key]).replace("|", "\|") for key in ["name", "type", "details"]]
        lines.append("| " + " | ".join(values) + " |")
    with open(path, "w", encoding="utf-8") as file:
        file.write("\n".join(lines) + "\n")


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--environment", required=True)
    parser.add_argument("--region", default="us-east-1")
    parser.add_argument("--output", default="documentation/output")
    args = parser.parse_args()
    os.makedirs(args.output, exist_ok=True)

    account, resources = collect_aws_resources(args.region)
    title = "BTSGSD AWS Infrastructure As-Built"
    metadata = {
        "Environment": args.environment,
        "AWS Account": account,
        "AWS Region": args.region,
```

```
        "Generated At": datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC"),
        "Total Resources": len(resources),
        "Inventory Scope": "Lambda and DynamoDB"
    }
    prefix = f"BTSGSD-{args.environment}-AsBuilt"
    create_word(os.path.join(args.output, prefix + ".docx"), title, metadata, resources)
    create_pdf(os.path.join(args.output, prefix + ".pdf"), title, metadata, resources)
    create_markdown(os.path.join(args.output, prefix + ".md"), title, metadata, resources)
    print("As-Built documentation generated successfully")


if __name__ == "__main__":
    main()
```

6 Azure DevOps pipeline

Create asbuilt-pipeline.yml at the repository root. Replace YOUR-AWS-SERVICE-CONNECTION with an authorized AWS service connection before running the pipeline.

```
name: BTSGSD-AsBuilt-Documentation

trigger: none
pr: none

parameters:
  - name: targetEnvironment
    displayName: Select Environment
    type: string
    default: dev
    values:
      - dev
      - prod

pool:
  vmImage: ubuntu-latest

steps:
  - checkout: self

  - task: UsePythonVersion@0
    displayName: Select Python
    inputs:
      versionSpec: '3.12'

  - script: |
      python -m pip install --upgrade pip
      python -m pip install boto3 python-docx reportlab
    displayName: Install Python Dependencies

  - task: AWSShellScript@1
    displayName: Generate Word and PDF As-Built
    inputs:
      awsCredentials: 'YOUR-AWS-SERVICE-CONNECTION'
      regionName: 'us-east-1'
      scriptType: 'inline'
      inlineScript: |
        set -euo pipefail

        python documentation/generate_asbuilt.py \
          --environment "${{ parameters.targetEnvironment }}" \
          --region "us-east-1" \
          --output "$(Build.ArtifactStagingDirectory)/asbuilt"

        ls -lh "$(Build.ArtifactStagingDirectory)/asbuilt"

  - task: PublishPipelineArtifact@1
    displayName: Publish Word PDF and Markdown
    inputs:
      targetPath: '$(Build.ArtifactStagingDirectory)/asbuilt'
      artifact: 'BTSGSD-AsBuilt-Documents'
      publishLocation: 'pipeline'
```

7 Run and retrieve artifacts

Create a pipeline from the existing YAML file, select dev or prod, and run it. The expected execution sequence is Python selection, dependency installation, AWS authentication, document generation, and artifact publication.

After a successful run, open Pipelines, select the run, and download the BTSGSD-AsBuilt-Documents artifact. It contains BTSGSD--AsBuilt.docx, .pdf, and .md.

1. Open Pipelines and select New Pipeline.

2. Choose Azure Repos Git and BTSGSD-Infrastructure.

3. Select Existing Azure Pipelines YAML file and /asbuilt-pipeline.yml.

4. Choose dev or prod and run the pipeline.

5. Download BTSGSD-AsBuilt-Documents from the completed run.

8 Publish Markdown as an Azure DevOps Wiki

Commit the generated Markdown under documentation/wiki, then use Overview > Wiki > Publish code as wiki. Select the BTSGSD-Infrastructure repository, the appropriate branch, and /documentation/wiki, and name the Wiki BTSGSD-AsBuilt.

The sample pipeline publishes Markdown as an artifact but does not commit it. Automated Wiki updates require a separate authenticated Git publishing step with appropriate branch permissions and approvals.

9 Extend the inventory

Once the baseline is stable, add Amazon Connect, Lex, VPC, IAM, CloudWatch, S3, and Terraform ownership details. Each extension should add only the required read APIs and should clearly record verification status and collection scope.

Service	As-built details
Amazon Connect	Instances, queues, flows, security profiles
Amazon Lex	Bots, aliases, versions
AWS Lambda	Names, runtimes, VPC configuration
Amazon DynamoDB	Tables, encryption, point-in-time recovery
Amazon VPC	Subnets, endpoints, security groups
AWS IAM	Roles and attached policies
Amazon CloudWatch	Log groups and alarms
Amazon S3	Buckets, versioning, encryption
Terraform	State ownership and module references

10 Recommendation

Keep this documentation pipeline separate from Dev and Prod deployment pipelines. Treat it as a read-only reporting workflow, retain generated files as pipeline artifacts, and review permission changes whenever inventory coverage expands.

Implementation checklist

- Create documentation/generate_asbuilt.py and asbuilt-pipeline.yml.

- Configure an authorized read-only AWS service connection.

- Run the pipeline for dev and validate all three artifacts.

- Repeat for prod after reviewing account and permission boundaries.

- Publish Markdown as code Wiki content when branch controls are approved.

- Extend one AWS service at a time and document every new read permission.